

Шпаргалка по кибербезопасности

Базовые принципы защиты, типовые уязвимости веб-приложений и меры против них, аутентификация и пароли, шифрование, безопасность зависимостей и контейнеров, DevSecOps, реагирование на инциденты.

Принципы

- **Наименьшие привилегии** — каждому ровно столько прав, сколько нужно для работы.
- **Многоуровневая защита** — один обходённый рубеж не должен открывать всё.
- **Безопасное состояние по умолчанию** — при сбое доступ закрывается, а не открывается.
- **Не доверять входным данным** — проверка на сервере обязательна, клиентская ничего не гарантирует.
- **Минимум поверхности атаки** — лишние порты, службы и учётные записи убираются.
- **Триада**: конфиденциальность, целостность, доступность — цели, которые нужно уравновесить.

Частые уязвимости веб-приложений

Нарушение контроля доступа

Пользователь получает чужие данные, подставив другой идентификатор. Проверяйте права на каждый объект на сервере

Внедрение SQL

Данные попадают в запрос как код. Лечится параметризованными запросами, а не экранированием вручную

Межсайтовый скриптинг (XSS)

Чужой сценарий выполняется в браузере пользователя. Экранирование вывода, шаблонизатор, политика CSP

Подделка межсайтового запроса (CSRF)

Действие выполняется от имени пользователя с другого сайта. Токен в форме и куки SameSite

SSRF

Сервер заставляют обратиться во внутреннюю сеть. Список разрешённых адресов, запрет на служебные подсети

Небезопасная десериализация

Разбор недоверенных объектов приводит к выполнению кода

Ошибки конфигурации

Открытые панели, отладочный режим, пароли по умолчанию, публичные хранилища

Аутентификация

- Пароли хранить только как хеши с солью: bcrypt, scrypt или Argon2 — не MD5 и не SHA.
- Длина важнее сложности; проверяйте пароль по спискам утёкших.
- Многофакторная проверка — самая результативная мера; ключи или приложение надёжнее кодов по SMS.
- Ограничивайте частоту попыток входа и делайте одинаковые сообщения об ошибке, чтобы не подсказывать существование учётной записи.
- Сессия: идентификатор в куке с флагами HttpOnly, Secure и SameSite; смена идентификатора после входа; разумный срок жизни.
- Маркеры доступа делайте короткоживущими, обновление — через отдельный маркер с возможностью отзыва.

Права доступа

Аутентификация ≠ авторизация

Первое — кто вы, второе — что вам можно

Проверка на сервере

Скрытая кнопка в интерфейсе не защищает конечную точку API

Ролевая и атрибутная модели

Права по ролям; права по признакам пользователя и объекта

Разделение обязанностей

Опасные операции требуют второго человека или подтверждения

Учёт действий

Журнал: кто, что и когда сделал; изменять его нельзя

Шифрование

В канале

TLS 1.2 и 1.3, HSTS, актуальные наборы шифров, автоматическое обновление сертификатов

На хранении

Шифрование дисков и резервных копий; отдельно — чувствительные поля в базе

Симметричное и асимметричное

Один общий ключ; пара из открытого и закрытого ключей

Хеш ≠ шифрование

Хеш необратим и служит для проверки, а не для хранения восстанавливаемых данных

Управление ключами

Хранилище секретов, ротация, разграничение доступа — ключ в репозитории обесценивает всю схему

Свои алгоритмы

Не изобретать: берите проверенные библиотеки и стандартные режимы

Секреты

- Никаких паролей и ключей в коде, конфигурации репозитория и журналах.
- Хранилище секретов или менеджер облака; в CI — маскирование и ограниченная область видимости.
- Сканер секретов в конвейере и на этапе коммита ловит случайные утечки.
- Утёкший ключ считается скомпрометированным навсегда: отзыв и замена, а не удаление коммита.
- Регулярная ротация и отдельные ключи для сред.

Зависимости и цепочка поставки

Файл блокировки версий

Воспроизводимая сборка; обновления осознанные, а не случайные

Проверка уязвимостей

Регулярный анализ зависимостей и автоматические обновления безопасности

Состав программного продукта (SBOM)

Перечень компонентов: позволяет быстро понять, затрагивает ли вас новая уязвимость

Проверка подлинности

Подписи артефактов и образов, доверенные реестры

Права в конвейере

Сборка не должна иметь доступа к рабочей среде; выкладка — отдельный этап с ограниченными правами

Контейнеры и инфраструктура

- Минимальный базовый образ, регулярное обновление, сборка в несколько этапов.
- Процесс не от имени root, файловая система только для чтения, отказ от лишних возможностей ядра.
- Секреты — через переменные среды исполнения или хранилище, не слоями образа.
- Сканирование образов на уязвимости в конвейере и в реестре.
- Сетевые политики: разрешено только то, что нужно; наружу открыт минимум.
- Инфраструктура как код с проверками конфигурации до применения.

DevSecOps

Смысл подхода — встроить проверки в обычный процесс разработки, чтобы дефекты находились рано и дёшево, а не на внешнем аудите перед релизом.

На этапе написания кода

Линтеры безопасности, проверка секретов, подсказки в редакторе

На запрос на слияние

Статический анализ, проверка зависимостей, обзор изменений затронутых мест

На сборке

Сканирование образов, подпись артефактов

На тестовом контуре

Динамическая проверка приложения, проверка конфигурации

В эксплуатации

Наблюдение, обнаружение аномалий, управление уязвимостями

Моделирование угроз

Разбор рисков на этапе проектирования — дешевле любой автоматике

Реагирование на инциденты

1. Подготовка
Контакты, роли, доступы, журналы, отработанный план
2. Обнаружение
Оповещения, обращения пользователей, внешние сообщения
3. Локализация
Изолировать затронутое, сохранить следы, не спешить с перезагрузкой
4. Устранение
Закрывать причину: уязвимость, доступ, ключ
5. Восстановление
Вернуть работу, наблюдать за повторением
6. Разбор
Без поиска виноватых: что позволило случиться и что менять в процессе

Заранее решите, кто уведомляет пользователей и регуляторов: сроки уведомления часто ограничены законом.

Личная гигиена

- Менеджер паролей и уникальный пароль для каждой службы.
- Второй фактор везде, где он есть, — в первую очередь на почте и в аккаунте разработчика.
- Обновления системы и браузера без отлагательств.
- Шифрование диска ноутбука и блокировка экрана.
- Внимательность к письмам со срочностью и ссылками на вход — так выглядит большинство успешных атак.
- Отдельные учётные записи для администрирования и повседневной работы.